Stable GPU sorting with recycled pages

An approachable guide to a merge sort that keeps the result in the original array and uses a small reusable workspace.

STABILITY REQUIRED

13 SEPTEMBER 2026 • IMPLEMENTATION 322ffcbec

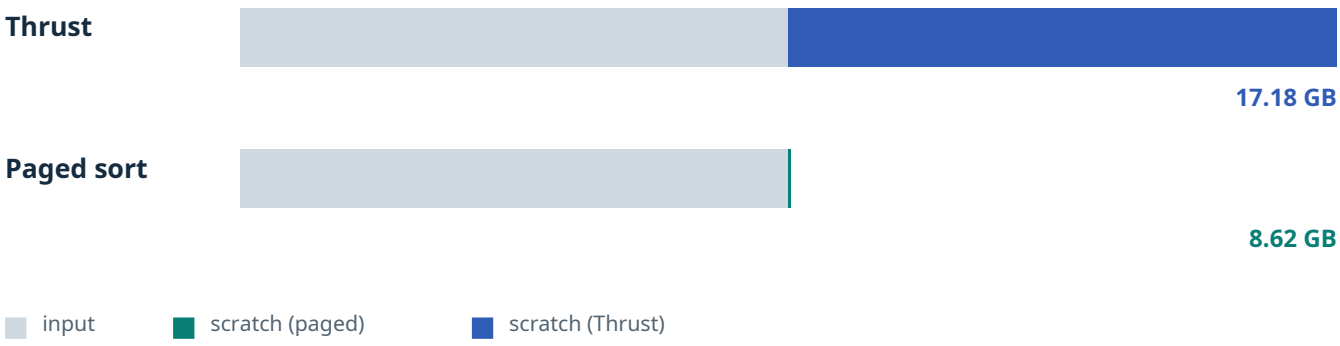
32.14 MB
temporary storage

86.2%
Thrust throughput retained

8.62 GB
accounted peak memory

Measured for **1,073,741,824 random 8-byte records** on one RTX PRO 6000 Blackwell. The comparison is against CCCL 3.4.2 through Thrust's stable comparison-sort path.

The memory difference



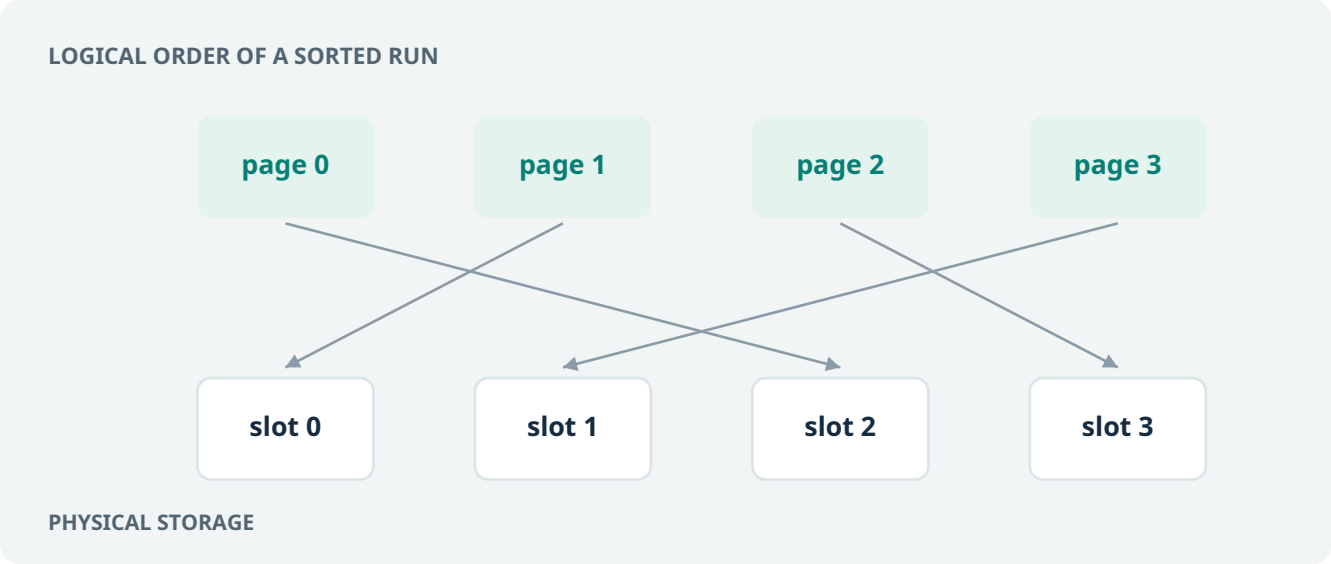
Peak = input + measured scratch. CUDA runtime/context and benchmark-only copies are excluded. Bars use a common linear scale; the 32 MB paged workspace is a thin sliver beside the 8.59 GB input.

ALGORITHM 02–04 RESULTS 05–06 VALIDATION & SOURCES 07–08

Keep the order in a page map

A run is a sorted sequence. A page is a fixed-size chunk of that sequence. The map tells us where each chunk currently lives.

The full-buffer comparison baseline allocates approximately another array of records while sorting. [2] This prototype keeps a small pool of spare pages instead. As merging consumes an input page, that storage becomes available for an output page.



Here the map is [2, 0, 3, 1]. Records remain contiguous *within* each page. These are software-managed chunks; the implementation does not remap hardware virtual memory.

1. **Sort locally**
   CUB stably sorts the initial shared-memory-sized runs.
2. **Plan each merge**
   Find input cuts for each output page and count its readers.
3. **Merge and recycle**
   Workers merge pages and release input storage after its last read.
4. **Swap the maps**
   Double the run size and repeat using the new logical order.
5. **Restore the array**
   Move pages into their final physical positions once, at the end.

Worker = CUDA thread block. The record benchmarks launch 376 blocks of 256 threads on 188 SMs. CUDA schedules the blocks; two per SM is an average, not a pinning rule.

Recycle only after the last read

Use four records per page for this illustration. The real benchmarks use thousands.

Two already-sorted runs occupy physical slots 0 and 1. Letters distinguish equal keys: 4a and 4b came before 4c in the original input. The comparator sees only the numeric key.

One possible execution order is shown below; actual workers can run concurrently.

LEFT RUN · SLOT 0

1	4a	4b	8

RIGHT RUN · SLOT 1

2	4c	7	9

Partition at output rank 4: take the first 3 records from the left run and the first 1 from the right. The remaining records form the second output page. Merge Path searches find such cuts without producing the entire merged array first. [1]

1 **Load the first output page**

Load [1, 4a, 4b] and [2] into shared memory. Both input pages still have one future reader, so neither is free. Write [1, 2, 4a, 4b] into spare slot 2.

2 **Load the second output page**

Load [8] and [4c, 7, 9]. Both reader counts now reach zero. Slots 0 and 1 can be recycled. Write [4c, 7, 8, 9] into slot 1, using the on-chip copy of its former contents.

3 **Put the pages in their final positions**

The output map is [2, 1]. Slot 0 is free, so copy output page 0 from slot 2 into slot 0. Output page 1 is already in place. The original array now contains the complete result.

FINAL STABLE ORDER

1	2	4a	4b	4c	7	8	9

Why ties stay stable: the initial sort is stable, the left run wins equal-key comparisons, and partition searches use the same rule. Moving whole pages preserves their logical sequence. The letters are explanatory labels, not extra indices required by the algorithm.

A small reserve keeps work moving

The reserve is tied to the number of workers. Metadata is tied to the number of pages.

Each worker processes a contiguous interval of output pages in order. Consumed input can be stranded at the two run boundaries, but the progress argument bounds this by two pages' worth per worker. It is this scheduling rule that makes the reserve predictable.

2P + 4

Spare pages when a complete output page fits in shared memory. Load first, release readers, then obtain the destination.

3P + 4

Spare pages for larger pages, which need several tiles. Their inputs remain protected until all tiles are consumed.

P counts launched worker blocks. The argument assumes eventual progress of CUDA work and atomic operations; it does not require every block to be resident at once. A kernel boundary separates successive merge levels.

Accounted storage across the random-record benchmarks

Records × size	Thrust scratch MB	Paged scratch MB	Thrust peak GB	Paged peak GB
16.8M × 8 B	134.25	24.91	0.268	0.159
67.1M × 8 B	537.00	25.26	1.074	0.562
268.4M × 8 B	2,148.01	26.63	4.295	2.174
1,073.7M × 8 B	**8,592.03**	**32.14**	**17.182**	**8.622**
67.1M × 16 B	1,074.00	25.71	2.148	1.099
268.4M × 16 B	4,296.02	28.47	8.591	4.323

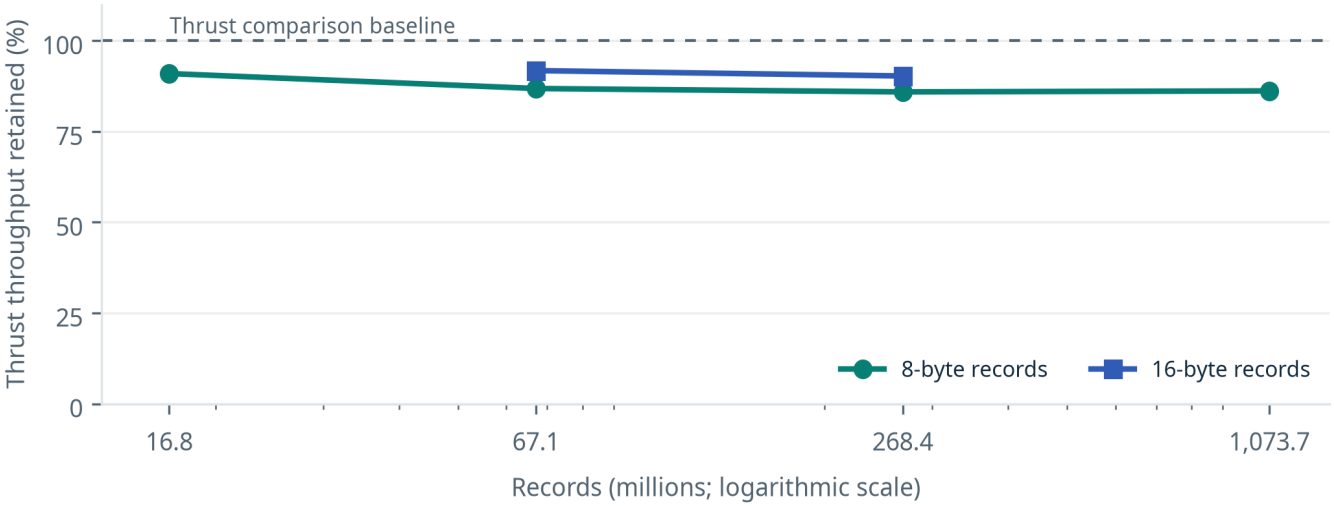
MB = 10^6 bytes; GB = 10^9 bytes. Peak = input + scratch. All explicit device workspace is counted, including maps, partitions, queues and locks. CUDA runtime/context and benchmark-only allocations are excluded.

Choosing a page size

With N records, B records per page and S bytes per record, scratch is **O(PBS + N/B)**. Larger pages trade more spare-record storage for less metadata. For fixed P and S, growing B with √N gives square-root growth; the prototype's 65,536-record cap eventually leaves a small linear metadata term.

Close to the comparison baseline

Random records, key-only comparison, and a complete stable result in the original iterator range.



Complete sort times

Records × size	Thrust ms	Paged ms	Throughput retained
16.8M × 8 B	2.61	2.87	91.0%
67.1M × 8 B	12.70	14.62	86.8%
268.4M × 8 B	57.98	67.49	85.9%
1,073.7M × 8 B	**258.31**	**299.73**	**86.2%**
67.1M × 16 B	27.20	29.63	91.8%
268.4M × 16 B	122.46	135.62	90.3%

Throughput retained = Thrust time / paged time. At 90% retained throughput, runtime is about 11% longer. The six random-record cases retain 85.9–91.8%; the largest 8- and 16-byte cases retain 86.2% and 90.3%, respectively.

HOW TO READ THESE NUMBERS

One RTX PRO 6000 Blackwell, 188 SMs; CUDA 12.9; CCCL 3.4.2. Values are medians of nine trials after five warmups per algorithm. CUDA-event intervals include the entire call, including host launch gaps and final page relocation. Input-reset copies and allocation growth are outside the timed interval.

Where the gains come from

The strongest refinement is less scratch. Removing fixed setup overhead also helps shorter sorts.

The revised implementation releases single-tile inputs earlier, shrinks the free-page queue to the worker reserve, and shares metadata between merge and relocation phases. It also replaces two full device-property queries with the needed SM-count query, removing about 1.6 ms of setup on this machine.

Records × size	Initial → refined ms	Initial → refined scratch MB
16.8M × 8 B	4.74 → 2.87	37.35 → 24.91
67.1M × 8 B	16.10 → 14.62	38.09 → 25.26
1,073.7M × 8 B	300.08 → 299.73	52.83 → 32.14
268.4M × 16 B	135.70 → 135.62	44.97 → 28.47

The initial and refined values come from separate nine-trial runs on the same GPU and CCCL version. Large-array runtime changes of about 0.1% are effectively unchanged here. Scratch falls by 33–39% across the six random-record cases.

Ordered inputs can bypass record merging

Already ordered run pairs retain their maps. Strictly separated pairs in reverse order can exchange page-map segments when alignment permits. These shortcuts explain the large speedups below; they should not be extrapolated to random data.

67.1M records	8-byte records	16-byte records
Random	0.87×	0.92×
17 distinct keys	0.87×	0.92×
All equal	5.88×	8.27×
Already sorted	5.87×	8.27×
Reverse ordered	0.89×	0.93×
Two separated runs	4.17×	5.70×
Two interleaving runs	3.02×	3.65×

Speed relative to Thrust; above 1× is faster.

Plain 32-bit keys remain a weaker case. At 67.1M keys, paged sort takes 7.82 ms: 74% of the comparison baseline's throughput and 21% of Thrust's default radix throughput. It uses 12.87 MB of scratch.

A working prototype, with clear limits

Stability and data preservation were checked against independent reference results.

3,658

correctness & stress cases

12

sanitizer configurations

The 8- and 16-byte suites compare complete records with CPU `std::stable_sort`, including the original position and payload. Cases cover ties, both comparator directions, tails and all supported page sizes. Stress cases exercise guarded subranges, workspace reuse, non-default streams, signed zeros and worker counts from 1 to 1,024.

Memcheck, Initcheck, Racecheck and Synccheck each passed three configurations covering early release and larger-page merging. Large benchmark outputs also match Thrust exactly. These checks cover finite inputs and executions, rather than proving every possible global-memory schedule.

Use the existing Thrust call shape

```
#include <thrust/sort.h>
#include "paged_stable_sort.cuh"

paged_sort::workspace scratch;
paged_sort::policy policy;
policy.scratch = &scratch;
thrust::stable_sort(policy.on(stream), first, last, comp);
```

The custom policy opts into the prototype. The call returns the complete result in [first, last) and synchronizes its selected stream. An optional reusable workspace avoids repeated allocation growth.

Current scope

Data types	Contiguous device iterators; trivially copyable, default-constructible records of at most 16 bytes.
Integration	Graph capture, asynchronous policies, allocator integration and concurrent workspace reuse still need production work.
Performance	One architecture and deterministic datasets. Very small workloads can have an unfavorable reserve; integer radix sorting remains much faster.
Traffic	Normal merge passes still read and write the records. The final page permutation adds O(N) record movement.

Enough detail to reproduce the result

The main explanation is complete. This page records the exact bound and the sources behind the study.

Symbols. N records; S bytes per record; B records per page; P worker blocks; Q = ⌈N/B⌉ pages. Let δ be 1 for a partial final page, else 0. Let E be 1 for early release, else 0; R = (3 − E)P + 4. K is the next power of two at least 3P + 4.

Exact auxiliary allocation, in bytes

$$(R + \delta)BS + 8Q + \max(20Q + 4(Q \bmod 2), 8(Q + R)) + 12K + 20$$

The terms are spare records, two page maps, the larger of the two phase-specific metadata layouts, the free queue, and counters. The shared phase region holds either reader counts plus partitions, or inverse ownership plus locks. Their lifetimes are separated by a kernel boundary.

Progress sketch for early release. If L tasks have loaded and released their inputs and D have acquired destinations, then D ≤ L. At least L − 2P input pages can be reclaimed. Free pages are therefore at least R + L − 2P − D ≥ R − 2P = 4. Worker caches may hold some of them. The full argument and the 3P + 4 case are in DESIGN.md.

Reproduction record

Implementation `322ffcbec`; initial prototype `a3df667d9`. CUDA 12.9.86, sm_120, driver 580.173.02. Measurements: 13 September 2026, nine-trial medians, five warmups, seed 42. Sorting used one GPU; the host reports two installed devices.

In `examples/low_memory_stable_sort`, README.md gives build commands; `run_validation.sh` and `run_benchmarks.py --full` reproduce the checks. `results/final.csv`, `scalar-final.csv` and `environment.json` contain raw measurements and exact source hashes. The original run is preserved separately.

Related work and primary sources

[1] O. Green, R. McColl and D. A. Bader. *GPU Merge Path: A GPU Merging Algorithm.* ICS 2012. The partitioning primitive used here.

[2] NVIDIA CCCL 3.4.2, *dispatch_merge_sort.cuh*. Baseline allocation and dispatch implementation; inspected in the pinned local checkout.

[3] F. Moltmann, T.-V. Nakajima and S. Wild. *Virtual-Memory Powersort.* arXiv:2605.27147v2, 2026. Related stable sorting with reusable pages.

Page reuse has prior art [3]. This study evaluates a GPU schedule, reclamation protocol and final parallel relocation; it makes no claim that the underlying page-recycling idea is new.